



UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Projeto de Software Web, Móvel e na Nuvem

Bancada

As ferramentas do projeto e porque foram usadas

Documento complementar ao relatório do projeto

Tiago Dias Pereira, n.º 55019

Universidade da Beira Interior

27 de setembro de 2026

Conteúdo

Resumo	3
1 Introdução	4
1.1 Porque este documento	4
1.2 Como ler as fichas	4
1.3 As três fases e quem usou o quê	5
2 A aplicação	5
2.1 Base de dados	5
2.2 API	6
2.3 Interface	9
2.4 Infraestrutura	11
3 Qualidade e verificação	12
4 Design	15
4.1 impeccable	15
4.2 taste-skill	18
4.3 awesome-design-md e o formato DESIGN.md	19
5 Documentação e relatório	20
5.1 Documentos em Markdown	20
5.2 Ferramentas do relatório	20
6 O assistente de programação: Claude Code	22
6.1 O que é e porque foi usado	22
6.2 Como trabalhou	22
6.3 As ferramentas do assistente	23
6.4 Ferramentas auxiliares	24
6.5 Quem decidiu o quê	24
6.6 Erros apanhados pelas verificações	25
6.7 Limites e transparência	25
7 Ferramentas das duas primeiras fases	26
8 Conclusão	27
Anexo A: Versões usadas	29
Anexo B: Comandos principais	29

Lista de Figuras

1	O processo de design com a impeccable.	16
2	Os pictogramas antes e depois da revisão de design.	17
3	O ciclo de trabalho da terceira fase.	22

Lista de Tabelas

1	Visão geral das ferramentas por área.	4
2	As decisões de produto e de design tomadas pelo autor.	17
3	As passagens do revisor de design.	18
4	Os documentos do repositório.	20
5	As ferramentas do Claude Code usadas na terceira fase.	23
6	O que foi decidido pelo autor e o que foi feito pelo assistente na terceira fase.	24
7	Alguns erros da terceira fase e a verificação que os apanhou.	25
8	Cada ferramenta numa frase.	27
9	Versões das ferramentas usadas na terceira fase.	29
10	Comandos principais do projeto.	29

Resumo

Este documento descreve, uma a uma, as ferramentas usadas para construir a Bancada, a aplicação de folhas de obra digitais do meu projeto final, e explica porque foram usadas. Cobre a aplicação em si (base de dados, API, interface e infraestrutura), as ferramentas de qualidade e verificação, as de design, as de documentação e o assistente de programação com inteligência artificial usado na última fase, o Claude Code.

Cada ferramenta aparece numa ficha com quatro campos: o que é, para que serviu neste projeto, porque foi a escolhida (e o que ficou de fora) e onde está no repositório. O documento separa o que escolhi nas duas primeiras fases do que entrou na terceira fase com o assistente. Mostra também, com números tirados do registo da sessão, como o assistente trabalhou e que erros as verificações apanharam pelo caminho.

1 Introdução

1.1 Porque este documento

A Bancada passou por três fases. Na primeira desenhei a base de dados. Na segunda comecei a API e a interface. Na terceira o projeto passou a servir várias oficinas, a interface foi refeita e ganhou testes automáticos, integração contínua e documentação. Ao longo disso entraram muitas ferramentas, umas escolhidas por mim desde o início e outras que só apareceram na terceira fase.

Numa apresentação de projeto final é natural perguntar: porquê esta base de dados? Porquê testes contra um SQL Server a sério? Para que serviu cada skill de design? O que fez o assistente de IA e o que decidi eu? Este documento responde a isso por escrito, com o mesmo cuidado do relatório principal: só descreve o que foi mesmo usado.

1.2 Como ler as fichas

Cada ferramenta tem uma ficha com o nome e a versão usada, e quatro campos:

O que é uma descrição curta da ferramenta;

Para que serviu o que fez neste projeto em concreto;

Porquê a razão da escolha e, quando existe, a alternativa que ficou de fora;

Onde está os ficheiros do repositório onde se vê o seu uso.

A Tabela 1 dá uma visão geral por área. No fim do documento, a Tabela 8 resume cada ferramenta numa frase, e os anexos têm as versões exatas e os comandos principais.

Área	Ferramentas
Base de dados	SQL Server 2022 (em Docker), driver <code>mssql</code> , <code>sqlcmd</code>
API	Node.js 22, Express 5, jsonwebtoken, cookie-parser, bcrypt, helmet, express-rate-limit, compression
Interface	React 19, Vite 8, React Router 7, vite-plugin-pwa, Fontsource (Barlow), Phosphor Icons (com um gerador próprio), pictogramas SVG próprios, CSS com tokens
Infraestrutura	Docker, Docker Compose, Dockerfile em etapas, ficheiros <code>.env</code>
Qualidade	<code>node:test</code> , <code>oxlint</code> , GitHub Actions, <code>npm audit</code> , GitGuardian, Playwright com Chromium, script de contraste
Design	impeccable, taste-skill (três skills), awesome-design-md, formato DESIGN.md
Documentação	Markdown, LaTeX (TeX Live, latexmk, TikZ), Graphviz, poppler-utils, Git e GitHub
Assistente de IA	Claude Code, com as suas ferramentas (terminal, ficheiros, subagentes, GitHub) e ferramentas auxiliares (Python, curl, openssl)
Fases 1 e 2	Arch Linux, VS Code (com a extensão SQL Server), Thunder Client, curl, ERDPlus, Docker, GitHub, LaTeX

Tabela 1: Visão geral das ferramentas por área.

1.3 As três fases e quem usou o quê

- **Junho de 2026 (primeira fase).** Levantei os requisitos a partir das folhas de obra da Duarte & Raposo, desenhei o modelo de dados no ERDPlus, pus o SQL Server a correr em Docker e liguei-lhe um primeiro servidor em Node.js.
- **Julho de 2026 (segunda fase).** Escrevi a API em Node.js com Express e testei-a à mão com o Thunder Client. Juntei o login com JWT e as primeiras proteções (helmet, limite de tentativas, bcrypt). Comecei a interface em React com Vite, React Router, o plugin da PWA e o oxlint. Estas escolhas foram minhas e mantiveram-se.
- **Setembro de 2026 (terceira fase).** Trabalhei com o Claude Code, um assistente de programação com IA que trabalha diretamente no repositório. As ferramentas novas desta fase foram usadas pelo assistente. As decisões de produto foram tomadas por mim, através de perguntas que ele me fez (secção 4.1).

Nas fichas, quando uma ferramenta já vinha das primeiras fases, isso fica dito no campo “Porquê”.

2 A aplicação

2.1 Base de dados

SQL Server 2022

imagem mssql/server:2022-latest

O que é O sistema de gestão de bases de dados relacional da Microsoft. A imagem oficial para Linux corre em Docker.

Para que serviu Guarda as seis tabelas da Bancada. Garante na própria base de dados várias regras do negócio:

- chaves estrangeiras compostas por oficina;
- índices únicos filtrados (a mesma matrícula só uma vez entre os veículos ativos de uma oficina);
- restrições CHECK para estados, cargos e categorias;
- DECIMAL exato para dinheiro;
- permissões por tabela: o login da API não pode apagar.

Porquê Foi a base de dados escolhida na primeira fase, e o modelo e o script já estavam feitos e testados. Na terceira fase manteve-se porque tem tudo o que a versão multi-oficina precisava. O DENY DELETE por tabela, em particular, garante o soft delete ao nível da própria base de dados. Em desenvolvimento, a imagem corre a edição Developer, a que vem por omissão (gratuita, mas só para desenvolvimento). Para uma oficina em produção, a edição Express é gratuita até 10 GB por base de dados. O PostgreSQL seria a alternativa óbvia (gratuito e com imagem para processadores ARM). Mudar teria obrigado a reescrever e voltar a validar o que já funcionava, sem ganho para o projeto.

Onde está backend/database/schema.sql, backend/scripts/db-setup.js, docker-compose.yml.

mssql (driver para Node.js)

12.7.2

O que é	O pacote que liga o Node.js ao SQL Server (usa o Tedious por baixo).
Para que serviu	Um único <i>pool</i> de ligações partilhado por toda a API. Consultas sempre parametrizadas (<code>.input(...)</code>) e transações para as operações que têm de ficar juntas, como criar a folha, o veículo e o cliente numa nova entrada.
Porquê	Já era usado na segunda fase e é o driver de referência para SQL Server em Node.js. Os parâmetros protegem contra injeção de SQL, e o <i>pool</i> evita abrir uma ligação nova em cada pedido.
Onde está	<code>backend/db.js</code> (<code>getPool</code> , <code>emTransacao</code>) e todas as rotas em <code>backend/routes/</code> .

sqlcmd

mssql-tools18

O que é	A linha de comandos do SQL Server, incluída na imagem Docker.
Para que serviu	Verificar se o SQL Server já está pronto: no <i>healthcheck</i> do <code>docker-compose.yml</code> e no arranque do SQL Server no CI.
Porquê	O SQL Server demora alguns segundos a arrancar. Sem esta verificação, a API e os testes tentavam ligar-se cedo demais e falhavam.
Onde está	<code>docker-compose.yml</code> , <code>.github/workflows/ci.yml</code> .

2.2 API

Node.js 22

22.22.2

O que é	O ambiente que corre JavaScript no servidor.
Para que serviu	Corre a API e os scripts da base de dados. Vários recursos que já vêm com o Node.js substituíram pacotes: • o executor de testes <code>node:test</code>; • o <code>node --watch</code>, que substitui o nodemon; • o <code>process.loadEnvFile</code>, que substitui o dotenv; • o <code>fetch</code>, usado pelos testes.
Porquê	A API já era em Node.js na segunda fase. A versão 22 é uma versão de suporte longo (LTS) e trouxe estas funções de base, o que permitiu tirar duas dependências (o nodemon e o dotenv) e dispensar o Jest e o Supertest, que o relatório intercalar sugeria. Menos dependências significa menos código de terceiros para rever e menos avisos do <code>npm audit</code> .
Onde está	<code>backend/package.json</code> (<code>engines</code> , <code>scripts</code>), <code>backend/lib/ambiente.js</code> .

Express 5

5.2.1

- O que é** Uma *framework* minimalista para servidores HTTP em Node.js.
- Para que serviu** Organiza a API em rotas (uma por recurso) e *middleware*: autenticação, verificação de origem, limites de pedidos e tratamento de erros. Em produção, serve também o frontend compilado, e é isso que põe a interface e a API na mesma origem.
- Porquê** Já era usado na segunda fase. A versão 5 passa sozinha para o tratamento de erros as exceções das rotas assíncronas, o que evita pedidos pendurados sem resposta.
- Onde está** backend/app.js, backend/routes/, backend/middleware/.

jsonwebtoken e cookie-parser

9.0.3 e 1.4.7

- O que é** Criação e verificação de JWT (*JSON Web Tokens*), e leitura de cookies nos pedidos.
- Para que serviu** A sessão é um JWT assinado com HS256, com emissor e audiência fixos, guardado num cookie `httpOnly` e `SameSite=Strict`. O tablet partilhado tem um segundo token, de dispositivo, com outra audiência, que não serve como sessão.
- Porquê** O JWT já existia na segunda fase, mas guardado no `localStorage`, onde qualquer script da página o pode ler. Passou para um cookie que o JavaScript não lê. Fixar o algoritmo e a audiência impede ataques com tokens de outro tipo ou sem assinatura, e há testes para isso.
- Onde está** backend/lib/sessao.js, backend/middleware/auth.js, backend/test/auth.test.js.

bcrypt

6.0.0

- O que é** Uma função de *hash* lenta, própria para guardar passwords.
- Para que serviu** Guarda as passwords e os PINs com custo 12. Também torna o login de tempo constante: compara sempre com um *hash*, mesmo quando o email não existe.
- Porquê** Já era usado na segunda fase. É o algoritmo mais conhecido para este fim e tem uma implementação nativa estável. O custo subiu de 10 para 12. O limite de 72 bytes do bcrypt passou a ser validado, em vez de ignorado em silêncio. O Argon2 seria uma alternativa mais moderna, mas o bcrypt com custo 12 continua adequado e não obrigou a migrar nada.
- Onde está** backend/lib/credenciais.js, backend/lib/validar.js.

helmet

8.3.0

O que é Um *middleware* que define cabeçalhos HTTP de segurança.

Para que serviu Define uma *Content-Security-Policy* rigorosa: scripts, estilos, fontes e imagens só do próprio site, e nada de código *inline*. Trata também dos outros cabeçalhos de proteção (*nosniff*, proteção contra *framing*, etc.).

Porquê Já era usado na segunda fase. A CSP é a segunda linha de defesa contra *cross-site scripting*. Foi por causa dela que as fontes deixaram de vir do Google e que o service worker da PWA passou a ser registado por um ficheiro à parte.

Onde está `backend/app.js`; teste em `backend/test/auth.test.js`.

express-rate-limit

8.7.0

O que é Um limitador do número de pedidos por intervalo de tempo.

Para que serviu Há vários limites:

- um limite geral para a API;
- no login, um limite por IP e conta, e um teto por IP;
- limites próprios para o registo de oficinas e para o PIN.

Porquê Já existia um limite no login, mas era só por IP. Numa oficina todos os dispositivos saem para a Internet com o mesmo IP, por isso um mecânico que se enganasse na password bloqueava a oficina inteira. A versão 8 traz o `ipKeyGenerator`, que trata bem os endereços IPv6.

Onde está `backend/middleware/seguranca.js`.

compression

1.8.2

O que é Compressão gzip das respostas HTTP.

Para que serviu Comprime as respostas da API e os ficheiros do frontend quando a API os serve.

Porquê É a forma mais simples de reduzir o que viaja pela rede, sem mexer no código das rotas. Conta sobretudo no tablet, ligado ao Wi-Fi da oficina.

Onde está `backend/app.js`.

O que saiu da API e porquê.

- **cors**: com a interface e a API na mesma origem, não é preciso abrir a API a outros domínios. Tirá-lo fecha uma porta.
- **dotenv**: substituído pelo `process.loadEnvFile` do Node.js 22.
- **nodemon**: substituído pelo `node --watch`.

2.3 Interface

React 19

19.3.0

- O que é** Uma biblioteca para construir interfaces a partir de componentes.
- Para que serviu** Todos os ecrãs da Bancada (quadro, modo bancada, nova entrada, folha de obra, gestão) e os componentes partilhados: tablier, matrícula, botões, campos, teclado do PIN.
- Porquê** Foi a escolha da segunda fase e é a biblioteca que aprendi. A interface foi refeita na terceira fase, mas continuou em React. Não se juntou nenhuma biblioteca de componentes: os componentes são próprios, o que dá controlo total sobre o aspeto e mantém o pacote pequeno.
- Onde está** `frontend/src/pages/`, `frontend/src/components/`.

Vite 8

8.3.1

- O que é** Uma ferramenta de desenvolvimento e compilação para aplicações web.
- Para que serviu**
- Servidor de desenvolvimento com recarregamento instantâneo, com um *proxy* que manda `/api` para a API. Assim, também em desenvolvimento, tudo parece vir da mesma origem.
 - Compilação para produção, com as páginas de gestão em ficheiros à parte.
- Porquê** Já era usado na segunda fase. É rápido e simples de configurar, e é hoje a escolha comum para projetos React sem servidor próprio de renderização.
- Onde está** `frontend/vite.config.js`.

React Router 7

7.18.4

- O que é** O sistema de rotas (endereços e navegação) para React.
- Para que serviu** As rotas da aplicação, a proteção das páginas que exigem sessão, e as páginas de gestão carregadas só quando são precisas. O browser adianta o carregamento dessas páginas quando está parado.
- Porquê** Já era usado na segunda fase. O carregamento à parte tira das mãos do tablet o código das páginas de gestão, que os mecânicos quase não usam.
- Onde está** `frontend/src/App.jsx`.

vite-plugin-pwa (Workbox)

1.3.0

- O que é** Um plugin que transforma a aplicação numa PWA: manifesto e service worker, gerado pelo Workbox.
- Para que serviu** A Bancada pode ser instalada no ecrã inicial do tablet e a aplicação fica guardada no dispositivo, para abrir logo. As respostas da API nunca ficam em cache, porque têm dados pessoais e têm de estar sempre atuais.
- Porquê** A PWA era um objetivo desde a primeira fase e o plugin já estava no projeto. Na terceira fase, o registo do service worker passou para um ficheiro à parte, porque a CSP não aceita código *inline*.
- Onde está** `frontend/vite.config.js`, `frontend/public/icons/`.

Fontsource (Barlow e Barlow Condensed)

5.3.0

- O que é** As fontes Barlow em pacotes npm, para servir a partir da própria aplicação.
- Para que serviu** A tipografia da Bancada: Barlow no texto e Barlow Condensed nos títulos, nas chapas de matrícula e nos números. Só se servem os ficheiros `woff2` dos conjuntos de caracteres latinos (`latin` e `latin-ext`).
- Porquê** Em julho as fontes (Oswald, Inter e IBM Plex Mono) vinham do Google Fonts. Isso trazia três problemas:
- mais ligações a outro servidor;
 - dados dos utilizadores a passar por terceiros;
 - incompatibilidade com a CSP rigorosa.
- A Barlow foi escolhida porque se inspira nas matrículas e na sinalética das estradas da Califórnia, o que casa com o ambiente de oficina e com as chapas de matrícula desenhadas na interface.
- Onde está** `frontend/src/styles/fontes.css`, `DESIGN.md` (secção Typography).

Phosphor Icons, com um gerador próprio

@phosphor-icons/core 2.1.1

- O que é** Uma coleção de ícones em SVG, com vários pesos (fino, normal, negrito, cheio...).
- Para que serviu** Os ícones de ações e de navegação: separadores, botões, telefone, tema, sair.
- Porquê** A biblioteca trazia seis pesos por ícone, cerca de 30 KB comprimidos, mais do dobro do código da aplicação. Um script (`npm run icones`) gera um módulo só com os ícones e os pesos usados (29 ícones), o que tirou 22 KB ao pacote inicial. O pacote fica só como dependência de desenvolvimento.
- Onde está** `frontend/scripts/gerar-icones.mjs`, `frontend/src/components/icones.js` (gerado).

Pictogramas próprios em SVG

Luzes.jsx

- O que é** Cinco pictogramas desenhados de propósito para os estados das folhas: carro, chave de bocas, pistão, bandeira de xadrez e chave.
- Para que serviu** As luzes do tablier, o painel do ecrã de entrada e o seletor de estado da folha de obra.
- Porquê** As luzes são a assinatura visual da Bancada e tinham de parecer luzes de aviso de um carro, e não ícones genéricos de aplicação. A geometria foi calculada com um pequeno script, numa grelha de 32×32 . O peso visual de cada símbolo foi medido no browser (entre 22% e 30% da grelha coberta), para nenhuma luz parecer mais fraca. A secção 4.1 mostra como mudaram com a revisão de design.
- Onde está** `frontend/src/components/Luzes.jsx`.

CSS com tokens

sem *framework*

- O que é** Folhas de estilo próprias, organizadas em variáveis CSS (*tokens*) para cores, tipos, espaços e raios.
- Para que serviu** Todo o aspeto da aplicação, com os temas claro e escuro. Os temas trocam os valores dos tokens, e há um pequeno script que aplica o tema escolhido antes de a página se desenhar.
- Porquê** Um sistema visual próprio (grafite, âmbar, luzes, chapas) fica mais simples com tokens do que com uma *framework* como o Tailwind ou o Bootstrap. O CSS fica em 8,4 KB comprimidos, e os tokens são a fonte de verdade que o DESIGN.md documenta.
- Onde está** `frontend/src/styles/tokens.css` e restantes ficheiros em `frontend/src/styles/`.

O que saiu da interface. As fontes do Google (Oswald, Inter, IBM Plex Mono) e os pacotes de tipos `@types/react` e `@types/react-dom`, que não tinham uso porque o projeto é em JavaScript e não em TypeScript.

2.4 Infraestrutura

Docker e Docker Compose

29.3.1 e v5.1.1

- O que é** Containers (ambientes isolados e reproduzíveis) e a ferramenta que descreve e arranca vários containers juntos.
- Para que serviu**
- O serviço `db` corre o SQL Server, com os dados num volume, uma verificação de saúde e a porta só em `127.0.0.1`.
 - O serviço `app`, num perfil à parte, corre a API a servir o frontend.
 - Na terceira fase, o Docker correu também os SQL Server de teste, incluindo um criado do zero para reproduzir o CI antes de abrir o *pull request*.

Porquê	Foi a minha escolha desde a primeira fase: o mesmo ambiente no meu computador, com Arch Linux, e em qualquer outro, incluindo um Mac com processador M (o <code>docker-compose.yml</code> já tinha essa configuração em julho), sem instalar o SQL Server na máquina. Na terceira fase a porta passou a estar só em <code>127.0.0.1</code> . Com <code>1433:1433</code> , o SQL Server ficava aberto a toda a rede, e o Docker contorna a <i>firewall</i> do sistema.
Onde está	<code>docker-compose.yml</code> .

Dockerfile em etapas

node:22-bookworm-slim

O que é	A receita da imagem de produção, construída em três etapas.
Para que serviu	A primeira etapa compila o frontend, a segunda instala só as dependências de produção da API, e a imagem final junta os dois, sem ferramentas de compilação e a correr com um utilizador sem privilégios.
Porquê	Uma imagem mais pequena e com menos superfície de ataque. Correr como <code>root</code> dentro de um container é um risco desnecessário. O <code>.dockerignore</code> deixa de fora segredos, testes, o relatório e as pastas de desenvolvimento.
Onde está	Dockerfile, <code>.dockerignore</code> .

Ficheiros `.env` e `.env.example`

configuração

O que é	Variáveis de ambiente num ficheiro que nunca vai para o git, e um modelo com todas as variáveis explicadas.
Para que serviu	Passwords do SQL Server, segredo das sessões, origens permitidas. A API recusa arrancar com um segredo curto ou copiado do exemplo.
Porquê	Nas duas primeiras fases, a password <code>sa</code> e um segredo JWT chegaram a estar no código, num repositório público. Continuam no histórico do git e nunca podem ser reutilizados. O <code>.env.example</code> , que o README já referia mas não existia, passou a existir.
Onde está	<code>.env.example</code> , <code>backend/config.js</code> .

3 Qualidade e verificação

node:test

Node.js 22

O que é	O executor de testes que vem com o Node.js.
Para que serviu	Os 52 testes automáticos da API, em cinco ficheiros. Arrancam a API a sério e fazem pedidos HTTP com cookies, como um browser, contra um SQL Server real, numa base de dados própria (<code>Bancada_Testes</code>) criada de novo em cada ficheiro. Testam: • o isolamento entre oficinas; • as sessões e os ataques a elas;

- as contas ao cêntimo;
- o modo bancada;
- a validação dos dados.

Porquê O relatório intercalar apontava o Jest e o Supertest como caminho. O `node:test` faz o mesmo sem dependências novas, e o `fetch` do Node.js substitui o Supertest. Testar contra um SQL Server real, e não contra uma imitação, foi uma escolha deliberada: as regras mais importantes (chaves compostas, CHECK, permissões) estão na própria base de dados e só assim são testadas.

Onde está `backend/test/` (a ajuda comum está em `ajuda.js`).

oxlint 1.85.0

O que é Um *linter* de JavaScript muito rápido, escrito em Rust.

Para que serviu Verifica o código do frontend antes de cada compilação e no CI. Apanhou, por exemplo, opções inválidas numa chamada ao `fetch`.

Porquê Já estava no projeto desde julho. Não precisa de configuração e corre em milissegundos, por isso não há desculpa para o saltar.

Onde está `frontend/package.json` (`npm run lint`),
`.github/workflows/ci.yml`.

GitHub Actions `actions/checkout@v7`, `actions/setup-node@v7`

O que é O serviço de integração contínua do GitHub.

Para que serviu Em cada *pull request* e em cada *push* para o `main` e o `dev`, correm dois trabalhos:

- o primeiro arranca um SQL Server num container, verifica as dependências da API com o `npm audit` e corre os 52 testes (cerca de 50 segundos no GitHub);
- o segundo verifica as dependências do frontend, corre o *linter* e compila a aplicação (cerca de 20 segundos).

Porquê Garante que nada entra no `dev` sem os testes passarem, numa máquina que não é a de quem escreveu o código. As passwords do SQL Server do CI são geradas na hora (com `openssl`) e nunca ficam escritas no repositório. As versões das *actions* foram confirmadas como as mais recentes antes de as fixar.

Onde está `.github/workflows/ci.yml`.

npm audit e lockfiles

npm 10.9.7

- O que é** A verificação de vulnerabilidades conhecidas nas dependências, e os ficheiros que fixam as versões exatas instaladas.
- Para que serviu** O `npm audit` corre no CI para a API e para o frontend. Verifica as dependências de produção e faz falhar o CI se houver alguma vulnerabilidade de gravidade alta ou crítica (neste momento não há vulnerabilidades conhecidas). Os `package-lock.json` garantem que todas as máquinas instalam exatamente o mesmo (`npm ci`).
- Porquê** Os *lockfiles* estavam no `.gitignore`, e cada instalação podia ter versões diferentes. Sem eles, o `npm ci` do CI nem sequer funciona.
- Onde está** `backend/package-lock.json`, `frontend/package-lock.json`, `.github/workflows/ci.yml`.

GitGuardian

app do GitHub

- O que é** Um serviço que procura segredos (passwords, chaves, *tokens*) nos commits.
- Para que serviu** Verificou os commits do *pull request* da terceira fase e não encontrou segredos.
- Porquê** Já estava instalada no repositório. Faz sentido num projeto em que um segredo já foi exposto: é uma verificação automática que não depende de alguém se lembrar.
- Onde está** nas verificações de cada *pull request* no GitHub.

Playwright com Chromium

1.63.0 e Chromium 141

- O que é** Uma biblioteca para controlar um browser a partir de código.
- Para que serviu**
- **Percurso completo da aplicação**, com a versão compilada servida pela API e a CSP ligada:
 - todos os ecrãs, no computador (1440 píxeis), no tablet deitado e ao alto, e no telemóvel;
 - temas claro e escuro;
 - o modo bancada com PIN;
 - o percurso escuta erros na consola e violações da CSP.
 - **Capturas**: as da revisão de design, as do README e as do relatório.
 - **Prova temporizada**: o quadro volta de uma folha com as luzes já acesas (128 ms depois do toque).
 - **Medição dos pictogramas**: o peso visual de cada um, contado num *canvas*.
- Porquê** Os testes da API não veem a interface. Um browser a sério apanha o que só se vê a correr, como botões com largura zero, violações da CSP ou texto cortado no telemóvel. Também permite ter capturas iguais sempre que se repete o percurso.

Onde está Os scripts ficaram na área de trabalho do assistente e não no repositório. As capturas estão em `docs/imagens/` e `Relatorio/Anexos/ecras/`.

Verificação de contraste

script

O que é Um pequeno script que calcula o contraste entre as cores de texto e de fundo, pela fórmula das WCAG.

Para que serviu Confirmar, nos dois temas, que o texto passa o nível AA (4,5:1) e que os elementos gráficos, como as bordas dos campos e os pontos de estado, passam 3:1. Na primeira verificação, a borda dos campos no tema escuro ficou abaixo do mínimo e foi corrigida.

Porquê Um tablet numa oficina é lido ao sol e com as lâmpadas da oficina, e um contraste fraco nota-se logo. Com um script, a verificação é feita às combinações todas e não a olho.

Onde está Os valores verificados são os de `frontend/src/styles/tokens.css`. O script ficou na área de trabalho do assistente.

4 Design

As ferramentas desta secção foram pedidas por mim: quando pedi o melhor design possível, indiquei a *impeccable*, a *taste-skill* e a coleção *awesome-design-md*. São *skills*: conjuntos de instruções, referências e scripts que um agente de IA segue para fazer uma tarefa de uma certa maneira. Foram instaladas no repositório com o comando `npx skills add` e as versões ficam registadas no `skills-lock.json`.

4.1 impeccable

impeccable

pbakaus/impeccable

O que é Uma *skill* de design para agentes de IA. Tem um processo com etapas, uma linha de comandos própria (*impeccable*), um detetor automático de problemas de design e um revisor independente, que corre como um segundo agente.

Para que serviu Conduziu todo o trabalho de design da interface, do registo do produto até ao `DESIGN.md` (Figura 1). A linha de comandos foi chamada 12 vezes:

- `context` (4), para carregar o contexto do produto;
- `concept-seed` (1), que sorteia as formas do ecrã principal que chegam ao autor;
- `surface-brief` (2), para escrever o contrato de direção do ecrã principal;
- `detect` (5), o detetor: da primeira vez deu um aviso, que foi corrigido, e no fim não encontrou problemas.

Porquê

Um assistente de IA sem processo tende a produzir interfaces genéricas, todas parecidas. A impeccable obriga a três coisas: fixar primeiro o produto e as decisões do autor, escrever uma direção antes de construir, e passar por um revisor que avalia capturas reais e pode mandar refazer. Sem essa última etapa, a primeira versão do tablier teria ficado. Antes de a correr pela primeira vez, o assistente leu o que a linha de comandos fazia: uma *skill* corre com as mesmas permissões do agente, por isso não se executa sem saber o que faz.

Onde está

.claude/skills/impeccable/, PRODUCT.md,
.impeccable/surfaces/, DESIGN.md, .impeccable/design.json.

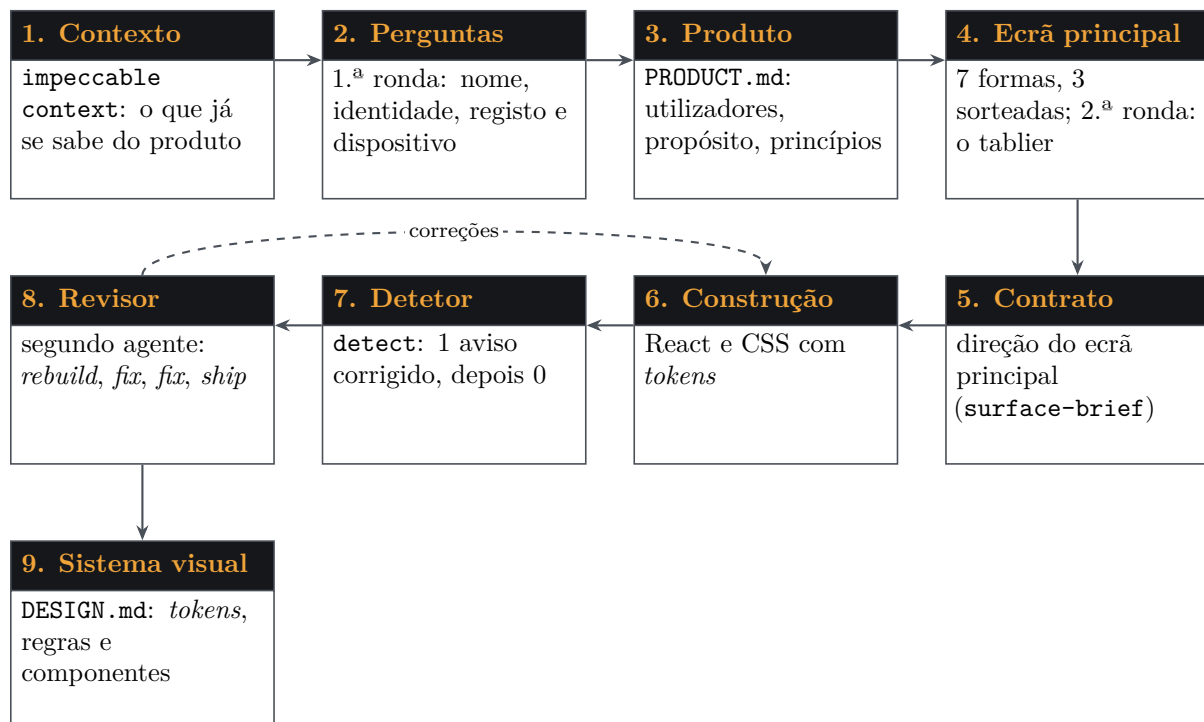


Figura 1: O processo de design com a *impeccable*, tal como foi seguido, pela ordem dos números. O revisor devolveu o trabalho à construção até dar as correções por resolvidas.

As decisões pedidas ao autor. A *impeccable* prevê uma página web onde o autor escolhe entre opções visuais. Como o assistente trabalhava num container na nuvem, sem uma página que eu pudesse abrir no meu browser, as decisões foram-me pedidas com perguntas estruturadas no próprio chat, com as opções e, no caso do ecrã principal, uma pré-visualização em texto de cada uma. Foram duas rondas, com as cinco decisões da Tabela 2. Em três perguntas (o nome, a identidade visual e a entrada de oficinas novas), o assistente marcou uma das opções como recomendada, e foram essas as que escolhi.

Antes da última pergunta, o assistente fez uma lista ordenada de sete formas possíveis para o ecrã principal: um quadro com uma coluna por estado, um quadro de chaves, começar pela matrícula, a lista e a folha lado a lado, o tablier de luzes, uma lista única com luzes e uma calha de folhas. As três opções que me chegaram não foram escolhidas por ele: um script da *impeccable* sorteia três posições da lista (saíram a 5, a 7 e a 4), para o autor ter uma escolha real e não só as preferidas do assistente.

Decisão	Opções apresentadas	Escolha
Nome da plataforma	Bancada; Oficina Digital; FolhaObra	Bancada
Identidade visual	manter e elevar; identidade nova; decide tu	manter e elevar (grafite, âmbar, luz indicadora)
Entrada de oficinas novas	registo público; só por convite	registo público
Onde os mecânicos usam a app	telemóvel; tablet na oficina; computador na receção	tablet na oficina
Ecrã principal do tablet	tablier de luzes; calha de folhas; lista e folha lado a lado	tablier de luzes

Tabela 2: As decisões de produto e de design tomadas pelo autor.

As passagens do revisor. O revisor é um segundo agente que não escreveu o código. Recebe as capturas dos ecrãs, o contrato de direção e o `PRODUCT.md`, e devolve um veredito com uma de quatro palavras:

- *recapture*: as capturas não servem;
- *rebuild*: refazer;
- *fix*: corrigir;
- *ship*: pode seguir.

A Tabela 3 mostra as quatro passagens, todas a 26 de setembro de 2026.

A Figura 2 mostra o efeito mais visível da segunda passagem: a roda dentada deu lugar a um pistão e as formas mais finas ganharam peso.



Figura 2: Os pictogramas dos estados antes e depois da segunda passagem do revisor. “A aguardar peças” deixou de ser uma roda dentada, e a chave de bocas, a bandeira e a chave ficaram mais cheias, para as cinco luzes terem um peso visual parecido.

Passagem	Veredito	O que pediu
1. ^a , revisão completa	<i>rebuild</i>	• refazer o tablier: as luzes estavam dentro de lentes redondas e pareciam botões; • matrículas com largura fixa e a faixa europeia; • todos os botões com 48 píxeis; • efeitos de rato só em dispositivos com rato; • ecrãs do tablet a seguir o tema; • botão de remover neutro em repouso; • luz de estado sem halo; • texto da pesquisa que não cabia no telemóvel.
2. ^a , revisão completa	<i>fix</i>	• a animação de arranque das luzes repetia sempre que se voltava ao quadro; • os pictogramas tinham pesos diferentes; • a roda dentada confundia-se com “Definições”; • no tablet ao alto os números não ficavam alinhados.
3. ^a , pontuação das correções	<i>fix</i>	Deu duas correções por resolvidas. Faltava a prova, numa captura, de que as luzes aparecem logo acesas ao voltar de uma folha.
4. ^a , pontuação da prova	<i>ship</i>	Captura feita 128 ms depois do toque, com as luzes acesas. Todas as correções dadas por resolvidas.

Tabela 3: As passagens do revisor de design.

4.2 taste-skill

taste-skill

Leonxlnx/taste-skill

O que é Um conjunto de três *skills* com regras para interfaces que não pareçam feitas em série por IA:

- **design-taste-frontend**, com as regras gerais;
- **redesign-existing-projects**, para melhorar um projeto que já existe;
- **stitch-design-taste**, para escrever o `DESIGN.md` num formato que outros agentes percebam.

Para que serviu As regras aplicadas na interface foram estas:

- um só acento de cor (o âmbar);
- fontes servidas pela própria aplicação;
- todos os estados de cada ecrã (a carregar, vazio, erro e normal);
- nada de ícones genéricos desenhados à mão (os pictogramas das luzes são próprios e pensados, o que é diferente);
- nenhum travessão nos textos da interface.

A parte de *redesign* levou a analisar primeiro o que existia (o ecrã de entrada de julho) antes de mudar: foi daí que ficou a luz indicadora.

Porquê	Pedi-a pelo nome. Complementa a impeccable: a impeccable dá o processo, a taste-skill dá regras concretas contra o aspeto genérico.
Onde está	.claude/skills/design-taste-frontend/ .claude/skills/redesign-existing-projects/ .claude/skills/stitch-design-taste/.

4.3 awesome-design-md e o formato DESIGN.md

awesome-design-md

VoltAgent/awesome-design-md

O que é	Uma coleção de ficheiros DESIGN.md inspirados nos sistemas visuais de produtos conhecidos.
Para que serviu	Referência de formato: como outros projetos organizam as cores, a tipografia, os componentes e as regras num DESIGN.md. Nenhum sistema visual de marca foi copiado: o da Bancada vem da oficina e da identidade que já tinha.
Porquê	Pedi-a pelo nome. Ver exemplos reais ajudou a escrever um DESIGN.md completo e no formato que as ferramentas esperam.
Onde está	Foi consultada, mas não faz parte do repositório.

Formato DESIGN.md

especificação da Google Labs

O que é	Um formato para descrever um sistema visual: um bloco YAML com os <i>tokens</i> (cores, tipografia, raios, espaços, componentes), seguido de oito secções em texto.
Para que serviu	O DESIGN.md da Bancada, com o sistema visual “O Tablier da Oficina”, as regras com nome (por exemplo, “A Regra da Voz Única”: o âmbar é a única cor de ação) e o que fazer e não fazer. O ficheiro .impeccable/design.json acrescenta o que o formato não guarda: rampas de cor, movimento e exemplos de componentes.
Porquê	Um sistema visual escrito deixa de depender da memória de quem o fez. Qualquer pessoa, ou agente de IA, que acrescente um ecrã tem as regras e os valores exatos à mão.
Onde está	DESIGN.md, .impeccable/design.json.

5 Documentação e relatório

5.1 Documentos em Markdown

A Tabela 4 resume os documentos do repositório. Três deles (`AI.md`, `CLAUDE.md` e `AGENTS.md`) existem a pensar em agentes de IA. Um agente que abra o repositório pela primeira vez, ou uma sessão que foi compactada, recupera assim o contexto sem ter de o adivinhar.

Documento	Para quê
<code>README.md</code>	apresentação do projeto, como o pôr a correr, estado, documentação e fluxo de trabalho
<code>PRODUCT.md</code>	o produto: utilizadores, problema, percurso de um carro pela oficina, o que existe e o que não existe (também lido pela <i>impeccable</i>)
<code>DESIGN.md</code>	o sistema visual, com os <i>tokens</i> e as regras
<code>AI.md</code>	mapa do código para agentes de IA e pessoas: regras que não se podem partir, API, comandos, glossário
<code>CLAUDE.md</code>	memória do projeto para o Claude Code: preferências do autor, decisões tomadas, ambiente, estado atual e registo por data
<code>AGENTS.md</code>	ponto de entrada para outros agentes (aponta para o <code>AI.md</code> e o <code>CLAUDE.md</code>)
<code>docs/analise.md</code>	a revisão de segurança, desempenho e viabilidade

Tabela 4: Os documentos do repositório.

5.2 Ferramentas do relatório

LaTeX (TeX Live e latexmk)

TeX Live 2023, latexmk 4.83

O que é Um sistema de composição de documentos a partir de texto com marcação.

Para que serviu O relatório do projeto e este documento. Pacotes usados:

- `babel`, para o português;
- `booktabs` e `tabularx`, para as tabelas;
- `listings`, para o código;
- `dirtree`, para a árvore do repositório;
- `acronym`, para os acrónimos;
- `hyperref`, para as ligações;
- TikZ, para os dois diagramas deste documento, desenhados com as cores da Bancada.

O `latexmk` repete a compilação até as referências cruzadas estarem certas.

Porquê O relatório já era em LaTeX desde a primeira fase. A composição tipográfica é melhor do que num processador de texto, e as referências a figuras, tabelas e secções atualizam-se sozinhas. O texto fonte fica no git, com histórico.

Onde está `Relatorio/*.tex` e os PDF compilados ao lado.

Graphviz 2.43.0

O que é Desenho de diagramas a partir de uma descrição em texto (linguagem DOT).

Para que serviu O esquema da base de dados da segunda versão e o diagrama da arquitetura, no relatório do projeto.

Porquê Os diagramas da primeira versão foram feitos no ERDPlus e ficaram como imagens, e na revisão da terceira fase foram encontrados neles dois erros. Com o Graphviz, o diagrama é texto: fica no git ao lado do esquema, revê-se como código e volta a desenhar-se com um comando quando o esquema muda.

Onde está `Relatorio/Anexos/*.dot`, com os PDF gerados ao lado.

poppler-utils 24.02.0

O que é Ferramentas de linha de comandos para PDF (`pdftopdf`, `pdftoppm`).

Para que serviu Converter as páginas dos relatórios em imagens, para rever a paginação uma a uma: páginas quase vazias, figuras longe do texto, títulos sozinhos no fim de uma página.

Porquê Um relatório que compila sem erros ainda pode ter má paginação, e isso só se vê a olhar para as páginas.

Onde está Não deixa ficheiros no repositório: serve só para rever.

Git e GitHub git 2.43.0

O que é Controlo de versões e o serviço onde o repositório está alojado.

Para que serviu

- **Ramos:** o `main` é a versão estável, o `dev` a integração, e cada tarefa tem um ramo próprio.
- **Commits** em português, a explicar o porquê de cada mudança.
- **Pull requests:** o da terceira fase entrou no `dev` com o CI verde.

Porquê Usei o GitHub desde o início. O ramo `dev` foi criado na terceira fase, a meu pedido, para haver mais organização: o que está no `main` só muda quando o `dev` está verificado.

Onde está https://github.com/TiagoPereira001/Projeto_final_UBI.

6 O assistente de programação: Claude Code

6.1 O que é e porque foi usado

O Claude Code é um assistente de programação com inteligência artificial, da Anthropic. Trabalha diretamente num repositório: lê e escreve ficheiros, corre comandos num terminal, usa um browser e fala com o GitHub. Neste projeto correu num container na nuvem, com o repositório, o Node.js, o Docker e um Chromium instalados.

Usei-o na terceira fase porque o que pedi era grande e tinha pouco tempo:

- uma análise profunda de segurança, desempenho e viabilidade;
- transformar o projeto numa plataforma para qualquer oficina;
- refazer a interface com o melhor design possível;
- criar os documentos de design e o mapa para agentes;
- rever o README e atualizar o relatório.

Um assistente que consegue ler o repositório inteiro, pôr o SQL Server a correr, escrever testes e ver a interface num browser consegue fazer isso e provar que funciona. As decisões de produto continuaram a ser minhas (secção 4.1).

6.2 Como trabalhou

O trabalho seguiu sempre o mesmo ciclo (Figura 3): ler e planejar, alterar, testar, ver a interface no browser, rever o design, guardar e integrar. Nada era dado por feito sem passar pelas verificações.

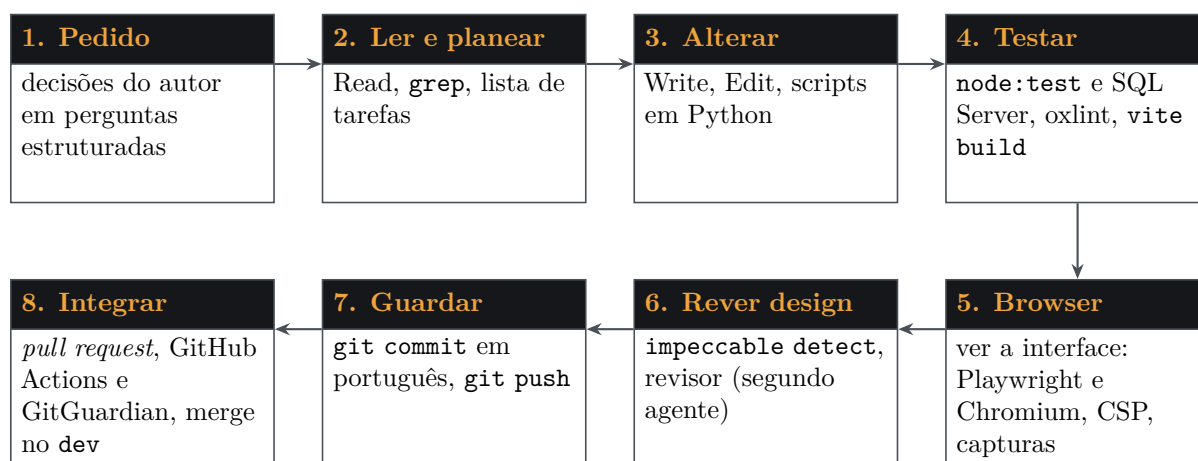


Figura 3: O ciclo de trabalho da terceira fase, com as ferramentas de cada passo, pela ordem dos números. Quando um dos passos de 4 a 8 falha, o trabalho volta ao passo 3.

6.3 As ferramentas do assistente

O Claude Code tem ferramentas próprias, que usa para agir no repositório. A Tabela 5 mostra as que foram usadas e quantas vezes. As contagens foram feitas com um script sobre o registo da sessão de trabalho, desde o primeiro pedido, a 26 de setembro de 2026, até ao pedido deste documento, a 27 de setembro. No total foram 510 chamadas.

Ferramenta	Vezes	Para quê
Bash (terminal)	267	correr testes, compilar, Docker, Playwright, git, scripts em Python, LaTeX, Graphviz
Write (escrever ficheiros)	98	criar ficheiros novos ou reescrever ficheiros inteiros
Read (ler ficheiros)	66	ler código, documentos, imagens das capturas e páginas dos PDF
Edit (editar ficheiros)	10	alterações pontuais (a maior parte das pontuais foi feita com scripts em Python)
TaskCreate e TaskUpdate	12 e 22	lista de tarefas visível, com o estado de cada uma
AskUserQuestion	2	as duas rondas de perguntas de decisão ao autor
Agent e SendMessage	1 e 3	o revisor de design: criado uma vez, retomado três vezes
ToolSearch	6	carregar ferramentas que só são ativadas quando são precisas
Ferramentas do GitHub	15	abrir o <i>pull request</i> , ler o estado do CI e fazer o merge no dev (a tentativa de ativar o merge automático falhou, porque essa opção não está ligada no repositório)
Avisos e agendamento	8	subscrever e deixar de subscrever os eventos do <i>pull request</i> , lê-los (4 vezes), agendar uma verificação e cancelá-la

Tabela 5: As ferramentas do Claude Code usadas na terceira fase.

Dentro do terminal, estas foram as ferramentas que entraram em mais comandos:

- scripts em Python (63 comandos) e o `git` (44);
- scripts do Playwright (28 comandos, 8 deles com o percurso completo da aplicação);
- o Docker (19) e as ferramentas de PDF do poppler (14);
- a compilação e o *linter* do frontend (13 e 12);
- o `latexmk` (12), o Graphviz (11) e o `curl` (11);
- a linha de comandos da impeccable (12 chamadas, em 9 comandos);
- os testes da API (9).

Um comando pode usar várias ferramentas ao mesmo tempo.

6.4 Ferramentas auxiliares

- **Python 3 (3.11), com PyYAML.** Serviu para alterações em muitos ficheiros de uma vez e para gerar ficheiros:
 - pôr as regras de `:hover` dentro de `@media (hover: hover)` em todas as folhas de estilo;
 - calcular a geometria dos pictogramas;
 - gerar o `.impeccable/design.json` a partir do `DESIGN.md`, com rampas de cor em OKLCH;
 - contar os números deste documento no registo da sessão.

Um script faz a mesma alteração em todo o lado e pode ser repetido.

- **curl.** Confirmar que a API estava de pé (`/api/saude`) antes de cada percurso no browser.
- **openssl.** Gerar passwords aleatórias para o SQL Server do CI e para os testes locais.
- **Docker (linha de comandos).** Arrancar, parar e apagar os SQL Server de desenvolvimento e de teste. No container na nuvem foi preciso arrancar o próprio serviço do Docker.

6.5 Quem decidiu o quê

A Tabela 6 separa o que foi decidido por mim do que foi feito pelo assistente na terceira fase. O código que ele escreveu passou pelos testes e pelo CI antes de entrar no `dev`.

Decidido por mim	Feito pelo assistente
• o nome Bancada; • tornar o projeto geral, com a Duarte & Raposo como primeiro cliente; • manter e elevar a identidade visual; • o registo público de oficinas; • o tablet partilhado como dispositivo principal; • o tablier como ecrã principal; • usar a <code>impeccable</code>, a <code>taste-skill</code> e a <code>awesome-design-md</code>; • criar o ramo <code>dev</code>, o <code>DESIGN.md</code>, o <code>AI.md</code>, o <code>CLAUDE.md</code> e o <code>PRODUCT.md</code>; • o relatório só com o que está feito; • o merge automático para o <code>dev</code> com o CI verde.	• a revisão de segurança, desempenho e viabilidade; • a segunda versão da base de dados e a API multi-oficina; • os 52 testes e o CI; • a interface nova e as correções pedidas pelo revisor; • os documentos, os diagramas e as capturas; • a atualização do relatório; • o <i>pull request</i>, o acompanhamento do CI e o merge.

Tabela 6: O que foi decidido pelo autor e o que foi feito pelo assistente na terceira fase.

6.6 Erros apanhados pelas verificações

O assistente também se engana, e é por isso que cada passo tem uma verificação. A Tabela 7 mostra alguns dos erros apanhados durante a terceira fase e a verificação que os apanhou.

Erro	Apanhado por
Os índices únicos filtrados falhavam por falta de <code>SET QUOTED_IDENTIFIER</code> ON no início do esquema.	criação da base de dados
Um teste usava 123456789 como exemplo de NIF inválido, mas esse número tem o dígito de controlo certo.	testes automáticos
Os botões só com ícone ficavam com largura zero, por causa de uma regra CSS para imagens.	percurso no browser
Os ícones Phosphor pesavam mais do dobro do código da aplicação.	medição do pacote
Os pontos do PIN tinham um brilho âmbar à volta, um efeito típico das interfaces geradas por IA.	detetor da impecable
No tema escuro, a borda dos campos tinha um contraste de 2,7:1, abaixo do mínimo de 3:1.	script de contraste
Uma chamada ao <code>fetch</code> com opções inválidas.	oxlint
Um teste de segurança dava falso positivo porque o nome da oficina de teste continha a palavra “Hashes”.	testes automáticos
As luzes do tablier estavam dentro de lentes e pareciam botões.	revisor de design (1. ^a passagem)
A animação de arranque das luzes repetia sempre que se voltava ao quadro.	revisor de design (2. ^a passagem)
A roda dentada de “a aguardar peças” confundia-se com “Definições”.	revisor de design (2. ^a passagem)
O Graphviz trocava as marcas das ligações entre tabelas lado a lado.	revisão das páginas do PDF

Tabela 7: Alguns erros da terceira fase e a verificação que os apanhou.

6.7 Limites e transparência

- **O assistente não substitui o teste com utilizadores reais.** A aplicação ainda não foi usada pelos mecânicos da Duarte & Raposo.
- **O veredito final do revisor de design (“ship”) tem um âmbito limitado.** Cobre as correções que foram pontuadas na última passagem, e não é uma aprovação nova de toda a interface.
- **Os dados das capturas e da demonstração são fictícios.** Os segredos do projeto (as passwords da base de dados e o segredo das sessões) nunca foram escritos no repositório.
- **O uso do Claude Code e das skills de design está declarado** na secção 4.5 do relatório do projeto e neste documento. As regras da UBI sobre o uso de IA em trabalhos académicos devem ser confirmadas por mim antes da entrega.

7 Ferramentas das duas primeiras fases

As duas primeiras fases foram desenvolvidas no meu computador, com Arch Linux, onde também corria o SQL Server em Docker.

Visual Studio Code

editor

O que é Um editor de código.

Para que serviu Escrever a base de dados, a API e o início da interface nas duas primeiras fases. A extensão SQL Server serviu para confirmar a ligação à base de dados e ver as tabelas.

Porquê É o editor que uso no curso, com extensões para JavaScript, React, Docker e SQL.

Onde está O ficheiro de *workspace* esteve no repositório em julho.

Thunder Client e curl

testes manuais

O que é Um cliente HTTP dentro do VS Code (semelhante ao Postman) e a ferramenta de linha de comandos para pedidos HTTP.

Para que serviu Os testes manuais da primeira API: criar e listar clientes, recusar autocaravanas de clientes inexistentes ou com matrícula repetida, confirmar que a password não aparecia nas respostas.

Porquê Eram a forma mais rápida de testar rota a rota enquanto a API crescia. Na terceira fase, estes cenários passaram para os testes automáticos.

Onde está Os cenários estão na Tabela 5 do relatório do projeto.

ERDPlus

diagramas

O que é Uma ferramenta web para diagramas entidade-relação e esquemas relacionais.

Para que serviu O diagrama concetual e o esquema relacional da primeira versão.

Porquê É simples e segue a notação usada nas aulas. Os dois erros que os diagramas tinham estão explicados no relatório. Na segunda versão, os diagramas passaram para o Graphviz.

Onde está Relatorio/Anexos/Diagrama.png,
Relatorio/Anexos/Esquema_relacional.png.

As outras ferramentas das primeiras fases (Docker, SQL Server, Node.js, Express, React, Vite, GitHub e LaTeX) continuaram na terceira fase e estão descritas nas secções anteriores.

8 Conclusão

As ferramentas da Bancada dividem-se em dois grupos. As da aplicação foram, na maior parte, escolhidas por mim nas primeiras fases e mantiveram-se: SQL Server, Node.js, Express, React, Vite e Docker. O que mudou nelas foi a forma de as usar: sessões em cookies, CSP rigorosa, porta do SQL Server só local, permissões mínimas, menos dependências.

As ferramentas novas da terceira fase servem sobretudo para duas coisas. A primeira é provar que o sistema funciona: testes contra um SQL Server real, integração contínua, percursos num browser a sério e um revisor de design independente. A segunda é deixar o conhecimento escrito: o `DESIGN.md`, o `PRODUCT.md`, o `AI.md`, o `CLAUDE.md` e os diagramas em texto. O assistente de IA fez grande parte do trabalho desta fase, mas as decisões de produto foram minhas, e o código que ele escreveu passou pelos mesmos testes e pela mesma integração contínua.

A Tabela 8 resume cada ferramenta numa frase.

Tabela 8: Cada ferramenta numa frase.

Ferramenta	Área	Porquê, numa frase
SQL Server 2022	base de dados	já escolhido e testado, e garante na própria BD as regras mais importantes
mssql	base de dados	<i>pool</i> único e consultas parametrizadas
sqlcmd	base de dados	saber quando o SQL Server está pronto
Node.js 22	API	versão LTS que substitui o nodemon, o dotenv e o Supertest
Express 5	API	rotas e <i>middleware</i> ; as exceções assíncronas chegam ao tratamento de erros
jsonwebtoken e cookie-parser	API	sessão num cookie que o JavaScript não lê
bcrypt	API	passwords e PINs com um <i>hash</i> lento e login de tempo constante
helmet	API	CSP rigorosa e cabeçalhos de segurança
express-rate-limit	API	limites por conta e por IP, sem bloquear a oficina inteira
compression	API	menos dados na rede do tablet
React 19	interface	a biblioteca já escolhida, com componentes próprios
Vite 8	interface	desenvolvimento rápido e compilação com páginas à parte
React Router 7	interface	rotas e páginas de gestão carregadas só quando são precisas
vite-plugin-pwa	interface	instalável no tablet e abre logo
Fontsource (Barlow)	interface	fontes da própria aplicação, compatíveis com a CSP
Phosphor + gerador	interface	ícones só com os pesos usados (menos 22 KB)
Pictogramas SVG próprios	interface	luzes de aviso que parecem as de um carro

Ferramenta	Área	Porquê, numa frase
CSS com tokens	interface	um sistema visual próprio em 8,4 KB
Docker e Compose	infraestrutura	o mesmo ambiente em qualquer computador
Dockerfile em etapas	infraestrutura	imagem pequena, sem <code>root</code>
<code>.env</code> e <code>.env.example</code>	infraestrutura	segredos fora do código
<code>node:test</code>	qualidade	52 testes contra um SQL Server real, sem dependências
<code>oxlint</code>	qualidade	<i>linter</i> instantâneo
GitHub Actions	qualidade	nada entra no <code>dev</code> sem os testes passarem
<code>npm audit</code> e <code>lockfiles</code>	qualidade	dependências verificadas e iguais em todas as máquinas
GitGuardian	qualidade	procura segredos em cada <i>pull request</i>
Playwright e Chromium	qualidade	ver a interface a funcionar, com capturas e medições
Script de contraste	qualidade	contraste do texto e das bordas confirmado nos dois temas
<code>impeccable</code>	design	processo com direção escrita, detetor e revisor independente
<code>taste-skill</code>	design	regras contra o aspeto genérico
<code>awesome-design-md</code>	design	exemplos do formato <code>DESIGN.md</code>
Formato <code>DESIGN.md</code>	design	o sistema visual escrito e reutilizável
Markdown (7 documentos)	documentação	contexto para pessoas e para agentes de IA
LaTeX e <code>latexmk</code>	relatório	composição de qualidade e texto fonte no git
Graphviz	relatório	diagramas como texto, fáceis de atualizar
<code>poppler-utils</code>	relatório	rever a paginação página a página
Git e GitHub	todos	ramos, <i>pull requests</i> e histórico
Claude Code	desenvolvimento	a terceira fase, com decisões do autor e verificação em cada passo
Python, <code>curl</code> , <code>openssl</code>	desenvolvimento	alterações em lote, verificações e passwords aleatórias
VS Code, Thunder Client, ERDPlus	fases 1 e 2	escrever, testar à mão e desenhar a primeira versão

Anexo A: Versões usadas

Ferramenta	Versão
Node.js e npm	22.22.2 e 10.9.7
SQL Server	2022 (mcr.microsoft.com/mssql/server:2022-latest)
Express, mssql, bcrypt	5.2.1, 12.7.2, 6.0.0
helmet, express-rate-limit	8.3.0, 8.7.0
jsonwebtoken, cookie-parser, compression	9.0.3, 1.4.7, 1.8.2
React, React Router, Vite	19.3.0, 7.18.4, 8.3.1
vite-plugin-pwa, oxlint	1.3.0, 1.85.0
@fontsource/barlow, @phosphor-icons/core	5.3.0, 2.1.1
Docker, Docker Compose	29.3.1, v5.1.1
GitHub Actions	actions/checkout@v7, actions/setup-node@v7
Playwright, Chromium	1.63.0, 141
TeX Live, pdfTeX, latexmk	2023, 1.40.25, 4.83
Graphviz, poppler-utils	2.43.0, 24.02.0
Python, git	3.11, 2.43.0

Tabela 9: Versões das ferramentas usadas na terceira fase.

Anexo B: Comandos principais

Comando	O que faz
<code>docker compose up -d</code>	arranca o SQL Server (só em 127.0.0.1)
<code>npm run db:setup</code>	cria a base de dados, as tabelas e o login da API
<code>npm run db:seed</code>	carrega a Duarte & Raposo com dados fictícios
<code>npm run dev (backend)</code>	API em <code>http://localhost:3000</code>
<code>npm run dev (frontend)</code>	interface em <code>http://localhost:5173</code>
<code>npm test (backend)</code>	os 52 testes contra o SQL Server
<code>npm run lint && npm run build</code>	<i>linter</i> e compilação do frontend
<code>npm run icones</code>	gera o módulo de ícones só com os pesos usados
<code>docker compose --profile app up -d --build</code>	a aplicação completa em containers
<code>latexmk -pdf <ficheiro>.tex</code>	compila um relatório
<code>dot -Tpdf <ficheiro>.dot -o <ficheiro>.pdf</code>	desenha um diagrama

Tabela 10: Comandos principais do projeto.